

Storage Wars

Every row has an address. Here's how your database finds it.

Issue #1

Thu, 01 Oct 2026

⌚ ~18 min read

Level 3/5

⚡ The short version

1. **Databases don't store "rows". They store fixed-size pages** (8 kB in Postgres, 16 kB in InnoDB), and an index is a tree of pages that points into them. Part 1 walks you from `SELECT` down to disk.
2. **PostgreSQL 19 pulled five features to ship on time.** Beta 4 is out, a release candidate is due in early October, and **Postgres 14 reaches end-of-life on 12 November.**
3. **MongoDB had a dramatic week:** its CEO left for Meta a day before Investor Day, where the company announced **MongoDB 9.0.** Databricks quietly shipped automatic change feeds and copy-free Iceberg sharing.

Release radar

18.6

current **PostgreSQL** minor; **19** is in beta, with an RC due early October

4.3

latest **Apache Kafka** feature line; **4.2.2** bug-fix shipped 29 Sep

12 Nov

PostgreSQL 14 end-of-life: no more fixes after this

Sources: postgresql.org versioning, PostgreSQL 19 Beta 4, Kafka blog.

PART 1 Learn



How a database stores a row: pages, heaps and the B-tree

FOUNDATIONS

~10 MIN

LEVEL 3

You've written `SELECT * FROM users WHERE id = 42` thousands of times. Do you know how many disk reads it costs, or why a random UUID primary key can make inserts slower? Almost every database performance conversation, from indexes and bloat to `VACUUM`, cache hit ratios and "why is this insert slow", comes back to one idea: **the page**.

THE UNIT OF EVERYTHING: THE PAGE

Disks and SSDs are slow compared with RAM, and they're happiest moving data in chunks. So relational databases never read or write a single row. They move **fixed-size pages**:

- **PostgreSQL:** 8 kB pages by default (set at compile time). ([PostgreSQL docs](#))
- **MySQL InnoDB:** 16 kB pages by default (set with `innodb_page_size` when the instance is initialised). ([MySQL docs](#))

A table is a file of pages. An index is another file of pages. A hot page lives in RAM in the **buffer pool** (Postgres calls it *shared buffers*), so the database can skip the disk entirely. That's why "does my working set fit in memory?" is the

first question in any database sizing discussion.

What happens under **SELECT ... WHERE id = 42**

Your query

planner chooses: index scan on users_pkey

Index (a B-tree of pages)

root → internal → leaf: finds the row's address

Buffer pool (RAM)

page already cached? done, no disk I/O

Data file (heap or clustered index)

reads the 8 kB / 16 kB page holding the row

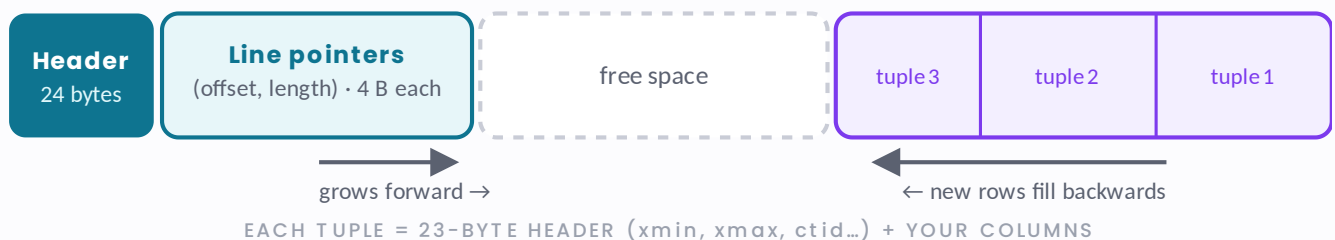
OS page cache → SSD

only reached on a cache miss

INSIDE A POSTGRES PAGE

Here's the part most developers never see. Each 8 kB Postgres page has a small **header (24 bytes)**, then an array of **line pointers (4 bytes each)** growing forward, then **free space**, then the actual **row versions ("tuples")** packed from the *end* of the page backwards. (PostgreSQL docs)

Anatomy of one 8 kB PostgreSQL heap page



Simplified: index pages also have a "special space" at the end. Sizes from the PostgreSQL page-layout docs.

Why the indirection? A row's address, its **ctid**, is (*page number, line-pointer number*). Because line pointers never move, Postgres can shuffle tuples around inside a page to compact free space without breaking any index that points at them. (PostgreSQL docs)

TWO PHILOSOPHIES: HEAP VS CLUSTERED INDEX

Here Postgres and MySQL take genuinely different paths, and it changes how you should design primary keys.

Where does the row actually live?

PostgreSQL: heap + separate indexes

- The table is a **heap**: rows go wherever there's free space
- **Every** index, including the primary key, is a separate B-tree whose leaves hold a `ctid`
- A lookup = walk the index → jump to the heap page

`users_pkey: 42 → (page 7, item 3)`

VS

InnoDB: the table IS the PK B-tree

- Rows are stored **inside the primary-key B-tree** (the clustered index)
- Secondary index leaves store the **primary key value**, not a physical address
- Lookup by email = email index → PK → clustered index

`idx_email: a@b.in → id 42 → row`

Sources: PostgreSQL storage docs, MySQL: clustered and secondary indexes.

MySQL's docs spell out the consequence: because every secondary index carries the primary key, "if the primary key is long, the secondary indexes use more space, so it is advantageous to have a short primary key." (MySQL docs)

THE B-TREE IN ONE PICTURE

A B-tree is a **balanced tree of pages**. Each internal page holds many sorted keys plus pointers to child pages, and the leaves hold the entries. Because one page holds many keys, the tree is very wide and very shallow. Every leaf is the same distance from the root, so a lookup costs a handful of page reads even for huge tables, and that cost grows **logarithmically** with table size. (Use The Index, Luke)

Finding id = 42 in a B-tree



Illustrative keys. Real trees are much wider: one internal page holds hundreds of keys, so even very large tables usually need only a few levels. And the top levels are almost always already in RAM.

In the real world: the random-UUID primary key

A very common pattern: an `orders` table in MySQL keyed by a random **UUIDv4**. Each new key lands at a random spot in the clustered B-tree, so inserts hit random pages all over the index instead of appending to the end. InnoDB's own docs describe the effect. With **sequential** inserts, pages fill to about **15/16**, but with **random** inserts they end up anywhere **between 1/2 and 15/16 full**. (MySQL docs) The result is more pages, more page splits, a bigger index competing for the buffer pool, and slower inserts as the table grows.

The fix that's become standard: **time-ordered IDs**. New keys sort near each other, so inserts append to the "right edge" of the tree. **PostgreSQL 18 added a built-in `uuidv7()`** for exactly this: a UUID whose leading bits are a millisecond timestamp. (PostgreSQL 18 docs) You keep globally unique IDs without the random-insert penalty.

🔥 Gotcha: in Postgres, an UPDATE is really an INSERT

Look at the tuple header again: every row version records `xmin` (the transaction that created it) and `xmax` (the one that deleted or replaced it). (PostgreSQL docs) When you `UPDATE` a row, Postgres doesn't overwrite it. It writes a **new tuple** and marks the old one dead. Dead tuples stay on the page until **VACUUM** cleans them up. Skip that and a heavily updated table "bloats": it gets bigger and slower even though the row count hasn't changed. One mitigation is the table's `fillfactor` (default 100, meaning pages are packed full). Lowering it to around 80–90 on update-heavy tables leaves room for the new version on the *same* page, which makes cheap "HOT" updates more likely. (PostgreSQL docs) MVCC and VACUUM get their own issues soon.

🔍 Under the hood

- **The 23-byte tuple header** holds `t_xmin`, `t_xmax`, `t_cid`, a 6-byte `t_ctid` pointing to the newer version of the row (if any), flags and the offset to your data. That's why a table of tiny rows is mostly overhead. (PostgreSQL docs)
- **InnoDB with no primary key** quietly picks the first `UNIQUE NOT NULL` index, and failing that generates a hidden clustered index on a synthetic row-ID column (`GEN_CLUST_INDEX`). You don't control it, so always declare a primary key. (MySQL docs)
- **InnoDB merges pages** when they drop below `MERGE_THRESHOLD` (50% by default), shrinking the tree after deletes. (MySQL docs)

🔧 Try it (15 minutes, Docker)

```
docker run -d --name pg -e POSTGRES_PASSWORD=pw postgres:18
docker exec -it pg psql -U postgres
```

```
CREATE TABLE t (id int, note text);
INSERT INTO t SELECT g, 'row ' || g FROM generate_series(1, 5) g;
SELECT ctid, * FROM t;                -- (0,1) ... (0,5): page 0, items 1-5
UPDATE t SET note = 'changed' WHERE id = 2;
SELECT ctid, * FROM t;                -- id 2 now lives at (0,6): a new tuple!
CREATE EXTENSION pageinspect;
SELECT lower, upper, pagesize FROM page_header(get_raw_page('t', 0));
SELECT lp, t_xmin, t_xmax, t_ctid FROM heap_page_items(get_raw_page('t', 0));
```

In the last query, the old version of row 2 shows a non-zero `t_xmax` and a `t_ctid` pointing to (0,6). That's MVCC on the actual bytes. `lower` and `upper` are where the free space starts and ends. (`pageinspect` needs a superuser, which `postgres` is.) (pageinspect docs)

WHY IT MATTERS TO YOU



Day-to-day impact



Interview value



Production risk



USE IT TOMORROW: SEE THE PAGES YOUR QUERY TOUCHES

All of this is visible from SQL. In Postgres 18, `EXPLAIN ANALYZE` automatically reports **buffers**: how many pages a query found in cache (**hit**) and how many it had to fetch from storage (**read**). (PostgreSQL docs)

```
EXPLAIN ANALYZE SELECT * FROM users WHERE id = 42;
--  Index Scan using users_pkey on users ...
--    Index Cond: (id = 42)
--    Buffers: shared hit=4          <- 3 index pages + 1 heap page, all from RAM
```

The output above is illustrative, but the shape is what you'll see. **A point lookup should touch a handful of pages.** If the same query reports thousands of buffers, it isn't using the index you think it is. If `read` is high and `hit` is low, your working set doesn't fit in memory. Four habits that follow from today's model:

1. **Read Buffers: before you read timings.** Time varies with load; page counts don't lie.
2. **Pick primary keys deliberately.** Short and time-ordered (`bigint` identity, or `uuidv7()`) beat random UUIDs, especially in InnoDB, where every secondary index carries the key.
3. **Give update-heavy Postgres tables room to breathe** with `fillfactor` below 100, and watch for bloat.
4. **Size memory for the hot pages, not the whole database.** A 500 GB database with a 20 GB hot set behaves very differently from one that touches everything.

GO DEEPER

- [Database Page Layout](#) (PostgreSQL docs). The official byte-by-byte description of what you just saw.
- [Clustered and Secondary Indexes](#) (MySQL docs). The InnoDB side of the story, in two short pages.
- [The Balanced Search Tree \(B-Tree\)](#) (Use The Index, Luke). The clearest free explanation of B-tree indexes, for every major database.
- [pageinspect](#) (PostgreSQL docs). The extension behind today's exercise. Great for building intuition.

~10 min • Postgres chooses reliability over features, MongoDB shuffles its leadership, and Databricks makes change data free.

Top story 1 • PostgreSQL 19 cuts five features to ship on time

POSTGRES

RC EARLY OCTOBER

What happened. PostgreSQL 19 Beta 4 shipped on **24 September**, and it's notable for what it *removed*. The project reverted several headline features rather than ship them half-baked. A release candidate is planned for **early October**, and GA "may also occur in October." The release note puts it bluntly: "*PostgreSQL must be reliable.*"

(PostgreSQL 19 Beta 4)

PostgreSQL 19: what stayed and what got pulled

Reverted in Beta 4

- SQL/PGQ, property-graph queries in SQL
- Online enabling/disabling of data checksums
- Temporal `UPDATE / DELETE ... FOR PORTION OF`
- `ALTER TABLE ... MERGE / SPLIT PARTITIONS`
- `pg_get_role_ddl()` and other DDL helpers

VS

Still in PostgreSQL 19

- `REPACK` command
- `WAIT FOR` command
- `CREATE PUBLICATION ... EXCEPT`
- A new **autovacuum scoring system**
- Faster foreign keys; SIMD-accelerated `COPY FROM`; logical-replication improvements

Source: PostgreSQL 19 Beta 4 announcement. Full list in the release notes.

Why it matters. Two of the survivors hit everyday pain directly. **Autovacuum scoring** changes how Postgres decides which tables to vacuum first. That matters for exactly the bloat problem in today's gotcha. **REPACK** targets reclaiming space from bloated tables. And the cuts are a healthy sign: Postgres ships every year, on schedule, because it's willing to drop what isn't ready. The reverted features may come back in a later release.

The date to put in your calendar: PostgreSQL 14's final release is **12 November 2026**. After that it gets no bug or security fixes. Each major version is supported for five years, and 15 follows in November 2027. ([postgresql.org versioning](https://www.postgresql.org/versioning/))

 **What to do.** On 14? **Plan the upgrade now.** 17 or 18 are the safe targets today; managed services often take a while to offer a brand-new major. On 18? Point a staging copy at the 19 RC when it lands and run your test suite. Watch autovacuum behaviour on your biggest tables.

WHY IT MATTERS TO YOU



Day-to-day impact



Act-now urgency



Production risk



Top story 2 · MongoDB: a CEO exit, then MongoDB 9.0

MONGODB

VENDOR CLAIMS

What happened. On **28 September**, MongoDB said CEO **Chirantan "CJ" Desai** had stepped down, effective immediately, to take a senior role at **Meta**. **Dev Ittycheria**, who ran the company from 2014 to 2025, is back as interim CEO while the board searches. MongoDB reaffirmed its guidance. ^(BigDATAwire) The next day, at its Investor Day, the company announced **MongoDB 9.0** as generally available. It claims **up to 35% faster find-one queries and 30% faster update-one queries**, and it also launched **Atlas Infinite** (public preview, aimed at AI workloads) and an **Atlas Agent Engine** for agent governance and memory. ^(GuruFocus)

MongoDB 9.0 speed-ups, as claimed by MongoDB



Vendor figures from the Investor Day announcement ^(GuruFocus). No independent benchmarks yet. Check the official 9.0 release notes before upgrading.

Why it matters. Single-document reads and writes by key are the bread and butter of most MongoDB apps, so if the gains hold up, they're broad ones. The leadership change matters less to your code than to your roadmap. Watch whether the AI push (Atlas Infinite, Agent Engine) keeps its pace under an interim CEO.

WHY IT MATTERS TO YOU



Day-to-day impact



Act-now urgency



Top story 3 · Databricks makes change data automatic and Iceberg sharing copy-free

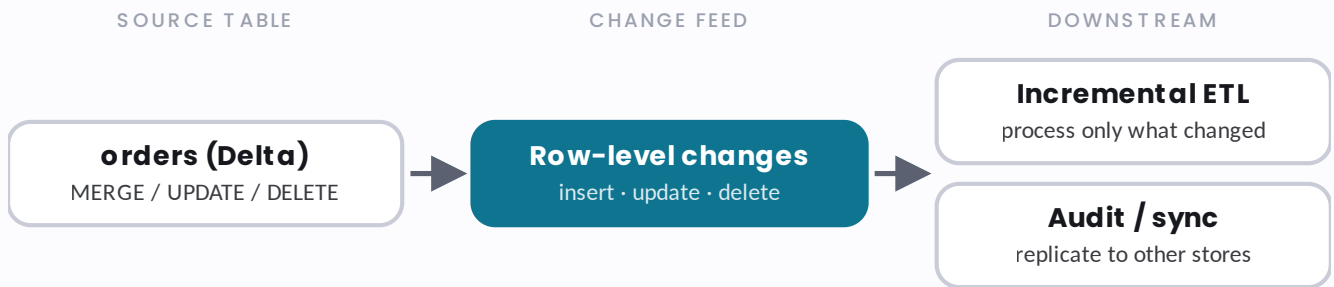
DATABRICKS

GA

What happened. Databricks' September release notes include several quiet but significant changes ^(Databricks release notes):

- **Automatic Change Data Feed (Auto CDF).** Row-level changes are computed at query time, with no need to enable CDF on each table first. Databricks says MERGE/UPDATE operations run about **15% faster**.
- **Share foreign Delta and Iceberg tables** via OpenSharing, without copying the data into Databricks first.
- **Query history system table (GA):** account-wide records of every query on SQL warehouses and serverless compute, with execution metrics.
- **Managed agent memory and sessions (beta),** backed by Lakebase, Databricks' Postgres-compatible operational database.

What a change data feed gives you



Change data capture comes up in depth later in the curriculum. The idea: downstream jobs read only the rows that changed, instead of rescanning whole tables.

Why it matters. Incremental pipelines are cheaper and faster than full rescans, but CDF used to be something you had to remember to switch on per table, *before* you needed it. Copy-free Iceberg sharing continues the industry's shift: **open table formats as the common layer**, with each engine (Databricks, Snowflake, ClickHouse, Fabric) querying the same files.

WHY IT MATTERS TO YOU



Day-to-day impact



Act-now urgency



⚡ Quick hits

📦 Kafka 4.2.2 is out (29 Sep).

It's a bug-fix release with "several critical fixes" for the 4.2 line. The newest feature line is 4.3, which shipped 25 KIPs in May, including tiered-storage follower bootstrap and share-group controls. (Kafka blog)

🏠 ClickHouse ≠ Microsoft Fabric (29 Sep).

A ClickHouse workload for Fabric is in public preview. Reading Iceberg tables in OneLake is now GA, and writing them back is in preview. (BigDATAwire)

🦆 DuckDB now ships inside dbt v2

(22 Sep). DuckDB 1.5.6 followed on 28 Sep, and a v2.0 preview, including a new parser, landed in August. (DuckDB news)

🔒 Redis 8.8.2 fixed 10 critical security issues

(August), including buffer overflows, use-after-free bugs and ACL bypasses. If you self-host 8.8, check you're patched. 8.8 also added a new array type and an `INCRBY` rate-limit command. (Redis release notes)

🚚 PostgreSQL Migrator 1.0

reached its first stable release (10 Sep). It's a community tool, listed in the Postgres news archive. (postgresql.org)

🇮🇳 PGConf India 2027:

dates are announced and the call for papers is open. Worth a look if you've got a Postgres war story. (postgresql.org)

📅 Coming up.

The PostgreSQL 19 RC (early Oct) and possibly GA (Oct), Postgres 14's end-of-life on 12 Nov, the DuckDB v2.0 release, and MongoDB's permanent CEO search.

Revision card

- Databases read and write **pages, not rows**: 8 kB in Postgres, 16 kB in InnoDB by default.
- The **buffer pool** caches hot pages in RAM. "Does the working set fit?" is the first sizing question.
- Postgres page: **24-byte header, 4-byte line pointers** growing forward, tuples packed from the end.
- A Postgres row's address is its **ctid** = (page, item).
- **Postgres = heap + separate indexes. InnoDB = the table is the PK B-tree**, and secondary indexes store the PK.
- **B-trees are wide and shallow**: a few page reads per lookup, growing logarithmically.
- **Random UUIDv4 PKs scatter inserts** (InnoDB pages $\frac{1}{2} - \frac{15}{16}$ full). Prefer time-ordered IDs; **PG 18 has `uuidv7()`**.
- **Postgres UPDATE = new tuple + dead old one** → VACUUM, bloat and **fillfactor**. **PG 14 is EOL on 12 Nov 2026**.

Storage Wars is researched and written by Claude every morning from official docs and release notes, project blogs and engineering write-ups. Vendor numbers are labelled as claims, and every item links to its source. Want more Kafka, less Mongo, or a different depth? Change `data/settings.yml` or just reply. 📧